Контест 2 · Сетки, игры и подписи

Неделя 4 · темы недель 3–4: поле игры как двумерный список, правила и проверка победы, клик по canvas, подпись `initData` и HMAC, Docker и Nginx. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею. Поле «4 в ряд» хранится как в лабе 4: `board[строка][столбец]`, строка 0 — верхняя, `0` — пусто, `1` и `2` — фишки игроков.

> Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

A. Сетки

A1. Верните фишки столбца `c` снизу вверх, пустые клетки пропустите.

```
Ввод:  column([[0, 0, 0],
               [1, 0, 2],
               [2, 1, 1]], 0)
Вывод: [2, 1]
```

A2. Включите «гравитацию»: все фишки падают вниз своего столбца, порядок фишек в столбце не меняется. Верните **новое** поле, исходное не трогайте. Подсказка: `[[0] * cols for _ in range(rows)]`, а не `[[0] * cols] * rows`.

```
Ввод:  gravity([[1, 0, 2],
                [0, 2, 0],
                [0, 0, 1]])
Вывод: [[0, 0, 0],
        [0, 0, 2],
        [1, 2, 1]]
```

A3. Найдите длину самой длинной непрерывной линии фишек игрока `p` по горизонтали или вертикали. Подсказка: столбцы поля — это `zip(*board)`.

```
Ввод:  longest_run([[1, 1, 0, 1],
                    [0, 1, 1, 1],
                    [2, 1, 2, 2]], 1)
Вывод: 3
```

B. Игры

B1. Обобщённые крестики-нолики: поле `n×n` задано строками из `X`, `O` и `.`. Побеждает тот, у кого `k` одинаковых знаков подряд по горизонтали, вертикали или диагонали. Верните победителя или `None`. Четыре направления — четыре пары `(dr, dc)`, а не четыре копии кода.

```
Ввод:  k_in_row(["X.O.", "XO..", "O...", "...."], 3)
Вывод: "O"
```

B2. Поле крестиков-ноликов — строка из 9 символов. Крестики ходят первыми. Чей сейчас ход: `"X"`, `"O"` или `"invalid"`, если такое число фишек получиться не могло.

```
Ввод:  next_turn("XO.X.....")
Вывод: "O"

Ввод:  next_turn("XXX......")
Вывод: "invalid"
```

B3. Canvas с полем `rows×cols` растянут на странице до `css_w×css_h` CSS-пикселей. По координатам клика внутри canvas (`event.offsetX`, `event.offsetY`) верните `(строка, столбец)` или `None`, если клик вне поля. Учтите, что на телефоне `css_w` не делится на `cols` нацело.

```
Ввод:  click_to_cell(175, 10, 350, 300, 6, 7)
Вывод: (0, 3)

Ввод:  click_to_cell(350, 0, 350, 300, 6, 7)
Вывод: None
```

B4*. Простой бот для «4 в ряд». Выберите столбец для игрока `p`: сначала тот, что выигрывает сразу; если такого нет, то тот, что закрывает немедленный выигрыш соперника; иначе свободный столбец, ближайший к центру (при равенстве левый). Если при выигрыше или защите подходят несколько столбцов, берите левый. Полное поле — `None`.

```
Ввод:  best_drop(<пустое поле 6×7>, 1)
Вывод: 3

Ввод:  best_drop(<внизу у игрока 2 фишки в столбцах 1, 2, 3>, 1)
Вывод: 0
```

C. Подписи

C1. Разберите строку запроса в словарь без `parse_qs` и `parse_qsl`. Пары разделены `&`, ключ и значение — первым `=`. `+` означает пробел, `%XX` — байты UTF-8 (можно `urllib.parse.unquote_plus`). Пара без `=` даёт пустое значение, пустые пары пропускаем.

```
Ввод:  parse_query("user=%D0%90%D0%BD%D1%8F&msg=hello+world&flag&a=1%3D2")
Вывод: {"user": "Аня", "msg": "hello world", "flag": "", "a": "1=2"}
```

C2. Напишите `sign(message, key)`, которая возвращает HMAC-SHA256 в hex. Затем по списку пар `(сообщение, подпись)` верните индексы подделок. Подпись может прийти заглавными буквами, это не подделка. Сравнивайте через `hmac.compare_digest`.

```
Ввод:  sign("id=42", "secret")
Вывод: "d04769d2311b20c5a04150ecc123192ce18ba3a6f2fc39d99d2a6e97abe5391e"

Ввод:  forged([("id=42", sign("id=42", "secret")), ("id=1", sign("id=42", "secret")),
               ("id=7", sign("id=7", "secret").upper()), ("x", "00")], "secret")
Вывод: [1, 3]
```

C3. Даны отметки `auth_date` (секунды), текущее время `now` и предел `max_age`. Сколько отметок свежие: не из будущего и не старше `max_age` секунд (ровно `max_age` — ещё свежая)?

```
Ввод:  fresh([100, 90, 200, 0, 40], 100, 60)
Вывод: 3
```

D. Docker

D1. Разберите строку из раздела `ports` файла compose в словарь `ip`, `host`, `container`, `proto`. Без IP публикуется на `0.0.0.0`; если указан только порт контейнера, то `ip` и `host` равны `None`. Протокол по умолчанию `tcp`.

```
Ввод:  parse_port("127.0.0.1:5432:5432")
Вывод: {"ip": "127.0.0.1", "host": 5432, "container": 5432, "proto": "tcp"}

Ввод:  parse_port("8080:80")
Вывод: {"ip": "0.0.0.0", "host": 8080, "container": 80, "proto": "tcp"}
```

D2. Разберите ссылку на образ в тройку `(реестр, имя, тег)`. Реестр — первая часть до `/`, если в ней есть `.` или `:` либо это `localhost`; иначе `docker.io`. Тег по умолчанию `latest`. Официальные образы Docker Hub получают префикс `library/`. Осторожно: в `localhost:5000/api` двоеточие — это порт, а не тег.

```
Ввод:  parse_image("python:3.12-slim")
Вывод: ("docker.io", "library/python", "3.12-slim")

Ввод:  parse_image("registry.example.ru:5000/team/bot:2.0")
Вывод: ("registry.example.ru:5000", "team/bot", "2.0")
```

D3. Разберите файл `.env` в словарь. Пустые строки и строки с `#` в начале пропускаем, `export` в начале убираем, строки без `=` пропускаем. Значение в кавычках (`"…"` или `'…'`) берём как есть без кавычек, а у значения без кавычек отрезаем комментарий `#…`.

```
Ввод:  export BOT_TOKEN=123:ABC
       DB_URL="postgresql://u:p@db/game"
       DEBUG=1 # включить
       HASH="a#b"
Вывод: {"BOT_TOKEN": "123:ABC", "DB_URL": "postgresql://u:p@db/game", "DEBUG": "1", "HASH":
"a#b"}
```

D4*. Возьмите `examples/Dockerfile.agent` из лабы 5 и перепишите его: образ `python:3.12-slim`, зависимости отдельным слоем до копирования кода, `.dockerignore`, запуск не от root, никаких секретов в образе. Покажите `docker images` (размер меньше 250 МБ) и вывод `report(...)` из лабы 5 на вашем Dockerfile: «Замечаний нет ✅».

E. Mini App и сервер

E1. `POST /check` с телом `{"init_data": "..."}` проверяет подпись `initData` по алгоритму Telegram (токен из переменной окружения `BOT_TOKEN`) и свежесть (не старше суток). Ответ — `{"ok": true, "user_id": 42}` или `{"ok": false, "user_id": null}`, даже на мусорную строку. Ответ не должен зависеть от порядка полей в строке.

```
Ввод:  POST /check {"init_data":
"auth_date=...&query_id=AAH&user=%7B%22id%22%3A+42...&hash=..."}
Вывод: {"ok": true, "user_id": 42}
```

E2. Страница с canvas: поле 6×7, при наведении подсвечивается столбец под курсором, клик бросает фишку (цвета игроков по очереди). Canvas чёткий на экране с `devicePixelRatio = 2`. Покажите в браузере и в Telegram.

E3. Соберите compose из двух сервисов: ваше приложение из E1 и `nginx`. Nginx отдаёт страницу из E2 по `/` и проксирует `/api/` на приложение. Наружу открыт только порт nginx. Покажите `curl localhost:8080/api/check ...`.

E4 · босс. «Сапёр»: класс `Minesweeper(rows, cols, mines)`, где `mines` — множество клеток `(r, c)` с минами. За каждую часть свой плюс.

а) `count(r, c)` — число мин среди восьми соседей. `view()` — список строк: `#` закрыто, `F` флаг, `.` открыто и рядом нет мин, цифра — открыто и рядом столько мин.

б) `open(r, c)` открывает клетку и возвращает, сколько клеток открылось. Если у клетки 0 соседей-мин, открываются и все соседи, и дальше волной (обход в ширину, `collections.deque`). Мина — `status = "lost"`, в `view()` мины показаны `*`. Открытая клетка или клетка под флагом — 0, ничего не меняется.

в) `flag(r, c)` ставит или снимает флаг на закрытой клетке и возвращает, стоит ли он теперь; `mines_left()` — мины минус флаги. Все клетки без мин открыты — `status = "won"`. После конца игры `open` и `flag` бросают `GameOver`.

```
Ввод:  g = Minesweeper(4, 5, {(0, 4), (2, 1)})
       g.open(3, 4)
       g.view()
Вывод: 9
       ["#####",
        "##111",
        "##1..",
        "##1.."]
```

E5*. API для «Сапёра» на FastAPI: `POST /games` (размер и мины, ответ 201), `POST /games/{id}/open` и `POST /games/{id}/flag` с телом `{"r": ..., "c": ...}`. В ответе `view`, `status`, `mines_left`, но **никогда** — список мин. Клетка вне поля — 422, нет игры — 404, игра окончена — 409. Тесты через `TestClient`.

```
Ввод:  pytest
Вывод: 4 passed
```